

CS201 Final

5 January 2026

Emre SEFER and Olcay Taner YILDIZ

Abstract—Write only ONE function per question in Java. Do not check the input, assume that the input is correct. Unless noted, (i) the input data structure is not empty, (ii) time complexity of the algorithm does not matter, (iii) you can not use extra data structures, (iv) you can use any function given in the data structure.

I. QUESTION (ALGORITHM ANALYSIS), 20 POINTS

```

1 int algorithm(int N){
2     if (N == 0)
3         return 0;
4     int sum = 0;
5     for (int i = 0; i < N; i++)
6         for (int j = 0; j < N; j++)
7             for (int k = 0; k < N; k++)
8                 sum++;
9     for (int i = 0; i < 4; i++)
10        for (int j = 0; j < 4; j++)
11            if ((i + j) % 2 == 0)
12                sum += algorithm(N / 2);
13            else
14                sum++;
15    return sum;
16 }
```

- 1) (10 Points) Construct the recurrence equation for this algorithm.
- 2) (10 Points) You are given another recurrence equation $T(n) = 4T(n/2) + O(n^2)$. Solve this without the master theorem.

II. QUESTION (LINKED LIST), 15 POINTS

Write the static method

`LinkedList pellNumber(int A, int B)`

that returns the Pell numbers between A and B (both inclusive) as a singly linked list. The first two Pell numbers are $Z_0 = 0$, $Z_1 = 1$, and the general formula is $Z_n = 2Z_{n-1} + Z_{n-2}$. Assume that $A \geq 2$ and $B > A$.

III. QUESTION (GRAPH), 15 POINTS

Write the method in linked list implementation

`int oddEdgeGraph()`

which returns the number of odd-valued edges, where an edge is odd-valued if its two distinct node values are both odd. You are not allowed to use extra data structures.

IV. QUESTION (TREE), 15 POINTS

Write a recursive method

`int quadraticSummation()`

in Treenode class that computes the sum of quadratic powers of all keys in a binary search tree.

V. QUESTION (SORTING), 15 POINTS

Given array A of N numbers write function

```
int[] minMaxRepeat(int[] A)
```

by using the existing quick sort implementation **quickSort**, which now orders the items as minimum item, maximum item, second minimum item, second maximum item, etc. You may assume list length is even. Input list should not be modified and you may use up to two extra arrays. For instance:

$[1, 8, 7, 3, 4, 5, 9, 2] \rightarrow [1, 9, 2, 8, 3, 7, 4, 5]$

VI. QUESTION (SORTING), 20 POINTS

Given an array A of N integers, write a function in `BucketSort` class

```
int secondLeastFrequent(int[] A)
```

which returns the second least frequent element in the given array A , which is already sorted. Your function must run in $O(N)$ time. You can not modify the input array A . You may use up to 1 external array.